

Sentry: Towards a Cross-Language Dynamic Analysis Framework for Hybrid Android Applications

Jyoti Prakash

The Mærsk McKinney Møller Institute
University of Southern Denmark
Odense, Denmark
jyp@mmmi.sdu.dk

Mathias Ransgaard Yde

The Mærsk McKinney Møller Institute
University of Southern Denmark
Odense, Denmark
mayde23@student.sdu.dk

Miguel Enrique Campusano Araya

The Mærsk McKinney Møller Institute
University of Southern Denmark
Odense, Denmark
mica@mmmi.sdu.dk

Abhishek Tiwari

The Mærsk McKinney Møller Institute
University of Southern Denmark
Odense, Denmark
abti@mmmi.sdu.dk

Abstract

Hybrid Android applications combine native components with web content rendered in WebView. Security-relevant behaviours can therefore cross the Android-JavaScript boundary through external page loading, bridge exposure, and browser-side operations. We present *Sentry*, a dynamic analysis framework that combines native instrumentation with a page-level JavaScript probe. Sentry injects instrumentation to record webview related properties from both native android side and WebView for analysis by the trace analyser. We evaluate Sentry on 18 benchmarks from the Iwandroid and HybriDroid benchmark suites. The results show that Sentry distinguishes calls to the original native bridge from JavaScript-side bridge mutations and preserves the ordering of bridge exposure, native updates, and invocation.

CCS Concepts

• **Security and privacy** → **Mobile platform security**; *Web application security*; • **Software and its engineering** → **Dynamic analysis**.

Keywords

Hybrid Android applications, WebView, dynamic instrumentation, Frida, JavaScript bridge, taint tracking, security reporting

ACM Reference Format:

Jyoti Prakash, Mathias Ransgaard Yde, Miguel Enrique Campusano Araya, and Abhishek Tiwari . 2026. Sentry: Towards a Cross-Language Dynamic Analysis Framework for Hybrid Android Applications. In *SECUTE 2026 (SECUTE '26)*, 2026, TBD. ACM, New York, NY, USA, 6 pages.

1 Introduction

Hybrid mobile applications combine web-based user interfaces with device-local functionality. On Android, this is commonly implemented through WebView, which renders web content inside a native application and can expose Android functionality to JavaScript code [5]. This arrangement improves reuse and development speed, but it also creates an execution boundary that is harder to analyse than either a purely native or a purely web-based application.

What makes these applications difficult to inspect is that the behaviours is split between the Android host and the embedded web page. The Android side decides which bridge objects, permissions, and APIs are exposed to embedded content, whereas the JavaScript side determines how those capabilities are used at runtime. Security-relevant behaviours may therefore emerge across WebView initialisation, JavaScript interface registration, and dynamically loaded pages. It may also depend on browser-side operations such as network requests, storage access, and dynamic code execution. Looking at only one side leaves an incomplete picture.

For example, consider a hybrid app that loads remote content into a WebView and exposes a JavaScript bridge for device functionality. Whether the app is vulnerable depends on more than the presence of the bridge alone. It also depends on which page is loaded, which methods are invoked, and how any returned data is used afterward. Existing analysis techniques offer only partial support for answering these questions. Static analysis can reason about framework usage and reachable code, but it is often challenged by dynamically loaded content, reflection, runtime code evaluation, and the semantic gap between Java/Kotlin and JavaScript. Prior work has shown both the prevalence of Android-web hybridization and the risks introduced by WebView bridges and code injection [10, 11, 13].

Sentry is a dynamic analysis tool for inspecting hybrid Android applications at the native-web boundary. It attaches to a running application using Frida [4], instruments Android framework methods related to WebView, and injects a lightweight JavaScript probe into the page context. In doing so, it captures behaviours on both the native side and the page side, including bridge registration and invocation, page loading, script evaluation, and selected JavaScript runtime events such as network requests and storage access. These observations are combined into a single trace that makes cross-layer interactions easier to inspect. Sentry is meant as an exploratory analysis aid for APKs. It runs without source code access or application repackaging in an emulator-based environment.

Sentry is intended to complement static analysis tools such as Iwandroid [14] and HybridDroid [7]. Those tools study hybrid applications through static analysis for information-flow control and bug detection [7, 14]. Sentry serves a different purpose. It shows which bridge objects are registered, which bridge methods are actually reached, and which browser-side actions occur during the

Listing 1: Android side of an illustrative hybrid-app pattern that exposes a privileged native bridge.

```
// Android side
webView.settings.javaScriptEnabled = true;
webView.addJavascriptInterface(new Bridge(), "native");
webView.loadUrl("https://example.com/app");

class Bridge {
    @JavascriptInterface
    public String getToken() { return readAuthToken(); }
}
```

same execution. Earlier work [13] has shown that WebView-based integration can introduce security risks. Static analysis can reveal exposed bridges and vulnerable patterns, but without dynamic instrumentation it remains difficult to determine whether those patterns are exercised at runtime and how they relate to browser-side behaviours. Sentry helps close that gap by logging bridge usage, host-side actions, and probe events from the embedded page, giving the analyst a concrete view of how the two sides interact during execution.

2 Background

Hybrid Android applications embed web content inside native code through WebView [5]. The Android host application controls WebView creation, configuration, and bridge exposure, while the embedded page executes JavaScript with its own browser-side APIs and state. Security-relevant behaviours are therefore often split across the native and web layers.

The main integration point is `addJavascriptInterface`, which exposes annotated Java methods to JavaScript [5]. Such native bridges can become dangerous when untrusted content is rendered in the WebView. Prior work has shown that WebView bridges, hybrid frameworks, and code injection can create exploitable attack surfaces [2, 10, 11]. Dynamic analysis is well suited to this setting because it can show which bridges are exposed, which methods are invoked at runtime, and which browser-side actions accompany those interactions. Sentry adopts this perspective and complements earlier static and large-scale studies of hybrid Android apps [7, 12, 14].

To make the discussion concrete, Listing 1 shows the native side of a common hybrid-app pattern in which JavaScript is enabled, a bridge object is exported under the name `native`, and the app loads content from a remote origin. In isolation, this code indicates potential risk, but it does not show whether the loaded page will actually invoke the bridge at runtime.

Listing 2 shows how the same pattern may unfold on the web side at runtime. Once the page gains access to the exported object, JavaScript can invoke a privileged bridge method and immediately forward the returned value to an external endpoint using `fetch`. This path spans page loading, bridge exposure, bridge invocation, and browser activity. Dynamic analysis can reveal this sequence during execution, which is precisely the kind of behaviours Sentry is designed to capture.

Listing 2: JavaScript side of the same pattern, invoking the native bridge and transmitting the result off device.

```
// JavaScript side
const token = window.native.getToken();
fetch("https://attacker.example/collect", {
  method: "POST",
  body: token
});
```

Purely static analysis is constrained because part of the relevant program may not be present in the APK at all. The WebView may fetch JavaScript at runtime, construct code strings dynamically, or execute browser-side logic through APIs such as `eval`, making reasoning across the Java/JavaScript boundary inherently incomplete. Dynamic instrumentation restricted to Java or Kotlin misses the other half of the behaviours. It can show bridge registration and framework calls, but not the page-side code that triggers them, the network requests that follow, or the storage accesses that accompany them. For hybrid apps, analysts therefore need observations from both the native host and the embedded page.

3 Sentry

Sentry observes how native Android code and embedded web content interact at runtime, then post-processes the resulting event stream into security findings. Its workflow has four main steps: attaching to the target process, instrumenting native WebView operations, injecting a lightweight page-level probe, and analysing the collected JSON events into a WebView security report.

Sentry targets hybrid Android applications in which Java or Kotlin code hosts a WebView, loads web content, and optionally exposes native functionality through `addJavascriptInterface`. Rather than proving the absence of vulnerabilities statically, it helps analysts answer concrete runtime questions: (1) which WebViews and settings are observed; (2) which origins are loaded; (3) which bridge objects and annotated methods are exposed to JavaScript; (4) which bridge methods are invoked at runtime and with what arguments or return values; (5) which WebView permissions are requested or granted; and (6) whether sensitive-looking bridge-return values reach JavaScript sinks such as network APIs, navigation, `postMessage`, `eval`, or storage.

3.1 Architecture

Figure 1 depicts Sentry as a four-stage analysis architecture comprising environment setup, native instrumentation, browser-side observation, and offline post-processing. The command-line wrapper stages the APK and package metadata, initializes the runtime services, records the analysis log, and invokes report generation after execution completes.

The runtime environment provides the execution substrate for the analysis. It comprises an Android emulator, a Frida server, a runner that attaches to the target application, and a demo web service used in the evaluation. After the target package has been launched, the instrumentation layer loads the hooks required for native-side observation and browser-side probing. One hook records WebView, WebSettings, bridge, WebClient, WebChromeClient, and

permission activity. The other ensures that the demonstration application resolves its URL target to the intended web origin.

The instrumentation layer yields two complementary event streams. Native events capture how the host application configures and mediates the WebView, while the injected JavaScript probe records browser-side activity within the page context. Both streams are written to a raw log and consumed by an offline analyser. The analyser reconstructs per-WebView state and generates structured reports containing findings, risk scores, WebView inventories, sensitive-flow evidence, and origin-to-bridge maps.

3.2 Native-Side Instrumentation

Sentry’s native instrumentation is implemented in the Frida WebView hook script. The script hooks the WebView constructor to assign stable WebView identifiers and enable WebView debugging. It hooks `getSettings` and selected `WebSettings` mutators to record security-relevant configuration changes, including JavaScript, DOM storage, file access, content access, file-origin access, mixed-content mode, Safe Browsing, geolocation, and automatic window creation.

For page and navigation behaviour, Sentry hooks WebView loading APIs and `WebViewClient` callbacks. It records explicit loads, posted URLs, inline HTML loads, base-URL loads, override decisions, intercepted requests, and page-completion callbacks. It also hooks `setWebViewClient` and `setWebChromeClient`. When an application installs a subclass, Sentry hooks that concrete client class as well. This is important because application-specific clients often mediate navigation, resource loading, permission prompts, and geolocation prompts.

For bridge behaviour, Sentry hooks `addJavascriptInterface`. When a bridge object is registered, Sentry records the WebView identifier, interface name, implementation class, current page URL, and origin. It then reflects over the bridge class and hooks only public methods annotated with `@JavascriptInterface`. Each invocation records the method signature, arguments, return value, errors, the origin at registration time, and the current origin at call time. Method names are matched against a taint heuristic covering terms such as camera, location, contact, token, cookie, credential, permission, file, device, and identifier-related names. Sensitive-looking methods are additionally prepared for JavaScript-side return-value tainting. In the current implementation, the tainting is not immune to program transformations on these terms, such as encoding the terms.

3.3 JavaScript-Side Instrumentation

Native hooks alone do not reveal the internal behaviour of the JavaScript runtime once content has been loaded. To address this limitation, Sentry injects a monitoring script into each observed WebView. The injected code creates a randomized namespace of the form `__sentry_webview_*`, stores events in a bounded page-local buffer, and wraps selected browser APIs with interceptors. The probe serializes arguments conservatively, bounds object traversal depth, and caps event buffers to avoid unbounded growth.

The JavaScript probe also implements lightweight taint propagation. When a sensitive-looking bridge method returns a value, Sentry attempts to wrap the corresponding JavaScript bridge method and remember a fingerprint of the returned value. The probe then

searches later sink arguments for that fingerprint, including nested arrays and objects and values reloaded from Storage. If a tainted value reaches a sink such as `fetch`, `XHR`, `sendBeacon`, `navigate`, `postMessage`, `eval`, or `Storage`, the resulting `js.probe` event carries taint metadata identifying the source bridge method and a bounded value preview. This mechanism is intentionally lightweight and evidence-oriented. It is designed to surface suspicious observed flows, not to provide a sound information-flow proof.

3.4 Trace Analyser

The trace analyser transforms the runtime trace into a WebView-centric security model. For each observed WebView, it associates loaded origins, security-relevant settings, exposed bridges and methods, bridge calls, browser-side operations, and evidence of sensitive data flows. It normalizes URLs to origins and compares them against a trusted-origin allowlist. This allows the analysis to relate a native bridge to both the content that received it and the actions that content performs.

A page load triggers a content finding when it uses cleartext HTTP, a local file, or inline HTML. A configuration change triggers a finding when it permits mixed content or enables JavaScript, file or content access, file-origin privileges, disabled Safe Browsing, geolocation, or automatic window creation. Registering a native bridge triggers a bridge-exposure finding. The finding has higher severity when remote content outside the trusted-origin allowlist receives the bridge. Exposing a method whose name matches the sensitivity heuristic triggers a sensitive-method finding. A bridge call from untrusted remote content triggers a bridge-invocation finding.

Permission requests and grants trigger permission findings. Native evaluation of JavaScript triggers an execution finding. Browser-side cleartext network use and dynamic code execution trigger JavaScript findings. Probe failures trigger health findings. Finally, a sensitive bridge return that reaches a network, navigation, messaging, storage, or dynamic-code sink triggers a taint-flow finding. Table 1 summarizes the resulting rule families and analyst-facing findings.

Across these rule families, findings are fingerprinted for deduplication, ranked by severity, and aggregated into a bounded risk score. The reports include a severity summary, per-finding evidence, a WebView inventory, sensitive-flow evidence, and an origin-to-bridge map.

3.5 End-to-End Analysis Workflow

Given a target APK, the analyst runs the analysis wrapper with the APK and, optionally, its package name. The script prepares the target APK and package name, starts the emulator and Frida services, launches the target under Sentry’s hooks, and streams the raw event log. When the run exits, Sentry automatically generates both JSON and Markdown reports. Because the APK is not modified, the same workflow can be reused across target packages with only package-name and environment adjustments. The resulting artifacts support two complementary inspection modes. These are low-level event replay from the raw JSON log and higher-level triage from the generated findings.

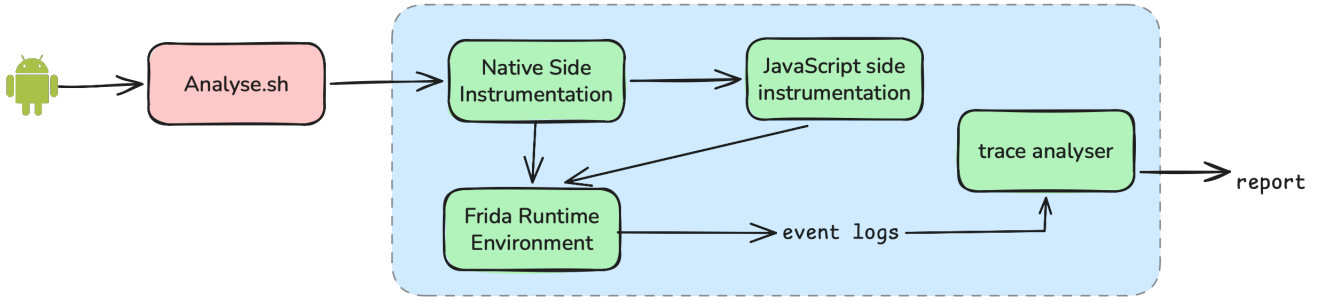


Figure 1: Sentry Architecture

Table 1: Rule families and analyst-facing findings.

Rule family	Reported finding
URL and loading	Cleartext WebView content; local file content; inline HTML.
Configuration	Misconfigured WebView settings.
Bridge exposure	JavaScript bridge exposed to WebView content.
Bridge methods	Sensitive-looking bridge method exposed.
Bridge invocation	Remote content called a native bridge method.
Permissions	WebView permission requested or granted.
Native JavaScript execution	Native code evaluated JavaScript in WebView.
JavaScript-side behaviour	JavaScript cleartext network use; JavaScript eval.
Probe health	JavaScript probe reported an error.
Taint-flow evidence	Sensitive native bridge data reached a JavaScript sink.

4 Evaluation

4.1 Preliminary Evaluation

We conduct a preliminary controlled evaluation of Sentry on 18 APK benchmarks. Eight *Iwandroid* [14] benchmarks test native bridge updates and JavaScript mutations. Ten *HybridDroid* [7] benchmarks cover bridge use, argument handling, and alias flows. Each benchmark defines the expected WebView and bridge behaviours. A run passes when its trace agrees with this oracle, including whether the original bridge is invoked and, where applicable, the order of bridge exposure, state update, and invocation. Table 2 reports the outcome for each benchmark.

All 18 benchmarks matched their expected observations. The results show that Sentry records bridge exposure and invocation, distinguishes calls to the original bridge from JavaScript-side mutations, and preserves the temporal context of native bridge updates.

This preliminary study evaluates controlled executions rather than complete applications. It does not measure coverage of remote origins, permission prompts, browser-side network activity, taint propagation, or runtime overhead. The results therefore provide evidence for the behaviours represented by the benchmark suites, not a general measure of coverage or soundness.

4.2 Proposed Evaluation

The planned study expands the corpus to benign and malware Android APKs. It includes applications with and without WebView use, native bridges, remote content, and browser-side network activity. To the best of our knowledge, there is no documented corpus for

Table 2: Preliminary benchmark results. A tick denotes agreement with the oracle.

Benchmark	Result
<i>Iwandroid benchmarks</i>	
strongUpdate	✓
strongUpdatecaseA	✓
strongUpdatecaseB	✓
strongUpdatecaseC	✓
JSUpdateCaseD	✓
JSUpdateCaseE	✓
JSUpdateCaseF	✓
JSUpdateCaseG	✓
<i>HybridDroid benchmarks</i>	
HelloCordova	✓
HelloHybrid	✓
HelloScript	✓
HelloScript22	✓
HelloScript_simple	✓
HelloScript_test-2	✓
HybridAPIArgNum	✓
NormalAliasFlowTest	✓
NormalAliasFlowTest_objfield1	✓
NormalAliasFlowTest_objfield_false	✓

the study of hybrid apps. Therefore, we envision to provide such corpus with reproducible ground truth established through Sentry. Documented interaction sequences and reproducible ground truth will distinguish absent behaviour from behaviour that exploration did not reach and will facilitate further research.

A common subset will support comparison with ω Test [6], Charlie [15], and Polycruise [8]. We will map outputs to shared cross-boundary objectives and classify them as true positives, false positives, or false negatives against the ground truth. We intend to

measure the behavioural coverage, true positives reported, and false negatives missed by the analysis.

Finally, we will assess Sentry alongside static analyses. For each baseline, precision and recall will be compared before and after incorporating runtime evidence. This will show whether dynamic observations remove false positives, expose missed cross-boundary behaviours, or both.

5 Related Work

Tiwari et al. proposed Charlie [15], which is a dynamic analysis framework targeted for fingerprinting WebView browsers. The work is orthogonal to Sentry as it is targeted for sound dynamic analysis and not necessarily for fingerprints. Hu et al. [6] introduced ω Test, a WebView-oriented testing technique that uses dynamic event-sequence generation to detect WebView bugs. However, like Charlie, its handling is limited to certain bugs. Polycruise [8] is a dynamic analysis framework for cross-language interactions, but it does not focus on information flow analysis of the WebView bridges. Similarly, ReUnify [9] performs cross-language static analysis for JavaScript and Java in the context of react native apps.

On the other hand, static analysis of WebView is also well-studied in literature. Yuan et al. [17] proposed WebViewJsdetect, which uses concurrent abstract interpretation to model the dynamic features of JavaScript within WebViews. Tiwari et al. [14] proposed a demand-driven information flow analysis for WebView bridges in Android hybrid apps with limited string resolution. Tiwari et al. also proposed LuDroid [13] for IFC analysis of hybrid apps, but it relies on coarse assumptions (e.g., treating any shared bridge function as leaking), which significantly reduces precision. In contrast, our approach is a dynamic analysis. Lee et al. [7] proposed HybriDroid, which constructs an interprocedural CFG for hybrid apps. We argue that it is difficult to analyse WebView communication with a pure static approach and an hybrid approach is a more promising direction. Bae et al. [1] formalized aspects of WebView semantics but focused on type-related vulnerabilities in bridge interfaces rather than information flow. Sentry is a first step towards a dynamic analysis that can be reused to improve the precision of static analyses.

Prior work has explored WebView security from multiple angles, including code-injection and cross-zone vulnerabilities [2, 11] and origin-related flaws [16]. Datta et al. [3] studied cross-boundary tracking via Java-to-JavaScript information diffusion, while Zhang et al. [18] focused on identity confusion vulnerabilities. In contrast, our work intends to develop a generalised solution for analysing WebView.

6 Conclusion

Sentry is a dynamic analysis framework for examining the Android-WebView boundary in hybrid applications. It combines native instrumentation with browser-side observation. The resulting trace records WebView configuration, page loading, bridge exposure and invocation, permission activity, and selected JavaScript operations. The trace analyser connects these observations to security findings, WebView inventories, sensitive-flow evidence, and origin-to-bridge maps.

The 18 benchmark results show that Sentry distinguishes calls to the original native bridge from JavaScript-side bridge mutations. They also show that it preserves the ordering of bridge exposure, native state updates, and invocation. These results provide evidence for the controlled behaviours exercised by the benchmark suites.

Like any dynamic analysis, Sentry reports concrete executions rather than all feasible paths. Its taint mechanism is evidence-oriented. It identifies observed bridge-to-sink flows but does not claim sound information-flow coverage. Future work will extend the evaluation to third-party applications, use WebView-aware exploration, measure runtime overhead, and replace value-fingerprint matching with stronger dynamic flow tracking.

Data Availability Statement. The Sentry tool developed in this paper is publicly available on GitHub at <https://github.com/jpksh90/sentry>.

7 Acknowledgements

This work is funded by Digital Research Centre Denmark (DIREC), National Defence Technology Centre (NFC) and Danish Industry Foundation (project 2025-0766).

References

- [1] Sora Bae, Sungho Lee, and Sukyoung Ryu. 2019. Towards Understanding and Reasoning About Android Interoperations. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 223–233. doi:10.1109/ICSE.2019.00038
- [2] Erika Chin and David Wagner. 2014. Bifocals: Analyzing WebView Vulnerabilities in Android Applications. In *Revised Selected Papers of the 14th International Workshop on Information Security Applications - Volume 8267 (Jeju Island, Korea) (WISA 2013)*. Springer-Verlag New York, Inc., New York, NY, USA, 138–159. doi:10.1007/978-3-319-05149-9_9
- [3] Sohoh Datta, Michalis Diamantaris, Ahsan Zafar, Junhua Su, Anupam Das, Jason Polakis, and Alexandros Kapravelos. 2026. Cross-Boundary Mobile Tracking: Exploring Java-to-JavaScript Information Diffusion in WebViews. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. San Diego, CA, USA.
- [4] Frida. 2026. Frida Documentation. <https://frida.re/docs/home/>. Accessed: 2026-05-06.
- [5] Google. 2024. Build web apps in WebView. <https://developer.android.com/develop/ui/views/layout/webapps/webview>. Accessed: 2026-02-17.
- [6] Jiajun Hu, Lili Wei, Yepang Liu, and Shing-Chi Cheung. 2023. ω Test: WebView-Oriented Testing for Android Applications. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023)*. ACM, 972–984. doi:10.1145/3597926.3598112
- [7] Sungho Lee, Julian Dolby, and Sukyoung Ryu. 2016. HybriDroid: static analysis framework for Android hybrid applications. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (Singapore, Singapore) (ASE '16)*. Association for Computing Machinery, New York, NY, USA, 250–261. doi:10.1145/2970276.2970368
- [8] Wen Li, Jiang Ming, Xiaopu Luo, and Haipeng Cai. 2022. PolyCruise: A Cross-Language Dynamic Information Flow Analysis. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2513–2530. <https://www.usenix.org/conference/usenixsecurity22/presentation/li-wen>
- [9] Yonghui Liu, Xiao Chen, Pei Liu, John Grundy, Chunyang Chen, and Li Li. 2024. ReUnify: A Step Towards Whole Program Analysis for React Native Android Apps. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (Echternach, Luxembourg) (ASE '23)*. IEEE Press, 1390–1402. doi:10.1109/ASE56229.2023.00113
- [10] Matthias Neugschwandtner, Martina Lindorfer, and Christian Platzer. 2013. A View to a Kill: WebView Exploitation. In *Presented as part of the 6th USENIX Workshop on Large-Scale Exploits and Emergent Threats*. USENIX, Washington, D.C. <https://www.usenix.org/conference/leet13/workshop-program/presentation/Neugschwandtner>
- [11] Claudio Rizzo, Lorenzo Cavallaro, and Johannes Kinder. 2018. BabelView: Evaluating the Impact of Code Injection Attacks in Mobile Webviews. In *Research in Attacks, Intrusions, and Defenses*, Michael Bailey, Thorsten Holz, Manolis Stamatogiannakis, and Sotiris Ioannidis (Eds.). Springer International Publishing, Cham, 25–46.
- [12] Abhishek Tiwari, Jyoti Prakash, Sascha Groß, and Christian Hammer. 2020. A Large Scale Analysis of Android – Web Hybridization. *Journal of Systems and Software* 170 (2020), 110775. doi:10.1016/j.jss.2020.110775

- [13] Abhishek Tiwari, Jyoti Prakash, Sascha Groß, and Christian Hammer. 2019. LUDroid: A Large Scale Analysis of Android – Web Hybridization. In *2019 19th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 256–267. doi:10.1109/SCAM.2019.00036
- [14] Abhishek Tiwari, Jyoti Prakash, and Christian Hammer. 2023. Demand-driven Information Flow Analysis of WebView in Android Hybrid Apps. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. 415–426. doi:10.1109/ISSRE59848.2023.00020
- [15] Abhishek Tiwari, Jyoti Prakash, Alimerdan Rahimov, and Christian Hammer. 2023. Understanding the Impact of Fingerprinting in Android Hybrid Apps. In *2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. 28–39. doi:10.1109/MOBILSoft59058.2023.00011
- [16] Guangliang Yang, Jeff Huang, Guofei Gu, and Abner Mendoza. 2018. Study and Mitigation of Origin Stripping Vulnerabilities in Hybrid-postMessage Enabled Mobile Applications. In *2018 IEEE Symposium on Security and Privacy (SP)*. 742–755. doi:10.1109/SP.2018.00043
- [17] Zhanhui Yuan, Zhi Yang, Jinglei Tan, and Hongqi Zhang. 2026. WebViewJSdetect: Javascript vulnerability detection in android webview via coverage-guided thread-adaptive concurrent abstract interpretation. *Computer Networks* 275 (2026), 111908. doi:10.1016/j.comnet.2025.111908
- [18] Lei Zhang, Zhibo Zhang, Ancong Liu, Yinzi Cao, Xiaohan Zhang, Yanjun Chen, Yuan Zhang, Guangliang Yang, and Min Yang. 2022. Identity Confusion in WebView-based Mobile App-in-app Ecosystems. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 1597–1613. <https://www.usenix.org/conference/usenixsecurity22/presentation/zhang-lei>